Legal Mind AI Terms & Conditions

A Project Report Submitted
In Partial Fulfillment of the Requirements
for the Degree of

MASTER OF COMPUTER APPLICATIONS

In

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

By

Chandan Boruah
(240231025)
Hari om Singh
(240231032)
Kunal Rastogi
(240231034)
Ritik Kumar
(240231050)

Under the Supervision of
Ms. Harshita Gupta
(Professor, CSE Department)
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
HARCOURT BUTLER TECHNICAL UNIVERSITY, KANPUR

to the

School of Engineering
Harcourt Butler Technical University, Kanpur
(Formerly Harcourt Butler Technological Institute, Kanpur)
Session: 2025-2026

TITLE OF THE PAGE

“Legal Mind AI Terms & Conditions”

ABSTRACT

Legal documents are often complex, lengthy, and difficult for non-experts to interpret, creating barriers to accessing legal knowledge and services. The increasing digitization of legal processes has opened opportunities to leverage Artificial Intelligence for simplifying legal understanding and improving decision-making. However, traditional methods of legal consultation remain time-consuming, expensive, and heavily dependent on human expertise.

This project introduces **LegalMind AI Terms & Conditions**, an intelligent legal document analysis system that utilizes advanced Natural Language Processing (NLP) techniques and Retrieval-Augmented Generation (RAG) to provide automated insights into legal documents. The system enables users to upload legal documents such as contracts, agreements, and policies, and receive structured outputs including summaries, clause-level explanations, and risk classification (High, Medium, Low).

The architecture of the system is built using a hybrid approach combining the MERN stack (MongoDB, Express.js, React.js, Node.js) with a Python-based AI processing layer. Legal documents are processed through text extraction, segmentation (chunking), and embedding generation, followed by storage in a vector database. During user interaction, relevant document segments are retrieved using semantic search, and a Large Language Model (LLM) generates context-aware responses grounded in the uploaded document.

In addition to AI-based analysis, the system incorporates a role-based framework consisting of Users, Lawyers, and Administrators. Users can interact with documents through a chat interface and request professional consultation, while verified lawyers can review cases and provide expert guidance through a dedicated dashboard. This hybrid approach ensures a balance between automated intelligence and human expertise.

The system is deployed through a responsive web interface, providing a seamless and user-friendly experience. The objective of LegalMind AI is not to replace legal professionals but to assist users by offering preliminary insights, reducing analysis time, and improving accessibility to legal services.

Overall, the project demonstrates the practical application of AI in the legal domain, highlighting its potential to transform how individuals and organizations interact with legal information. Future enhancements may include multilingual support, integration with legal databases and advanced AI models to further improve accuracy and scalability.

HARCOURT BUTLER TECHNICAL UNIVERSITY, KANPUR

CERTIFICATE

This is to certify that the project report entitled "LegalMind AI Terms & Conditions – AI-Powered Legal Document Analysis System" submitted by our team in fulfillment of the requirement for the award of the degree of Master of Computer Applications in the Department of Computer Science & Engineering, Harcourt Butler Technical University, Kanpur, is a record of original work carried out by us under the supervision of Ms. Harshita Gupta.

The matter embodied in this report has not been submitted elsewhere for the award of any other degree or diploma.

This is to certify that the above statements made by the candidates are correct to the best of our knowledge.

Date –

Ms. Harshita Gupta

(Professor, CSE department)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have supported us throughout the completion of our project, **LegalMind AI Terms and Conditions – AI-Powered Legal Document Analysis System**. This work has been a significant academic milestone, and its successful completion would not have been possible without the guidance and support of several individuals.

First and foremost, we are extremely thankful to our respected guide, **Ms. Harshita Gupta**, Department of Computer Science & Engineering, Harcourt Butler Technical University, Kanpur, for her invaluable guidance, continuous encouragement, and constructive feedback during every phase of the project. Her expertise and clear vision helped us remain focused on our objectives and successfully complete the project with clarity and purpose.

We would also like to express our sincere appreciation to the faculty members of the Department of Computer Science & Engineering for their valuable suggestions and academic support. The knowledge and skills gained through our coursework played a crucial role in the successful development of this project.

This being a group project, we would also like to acknowledge the collaborative efforts, dedication, and teamwork of all the members involved. Working together allowed us to share ideas, overcome challenges, and ensure smooth execution of the project.

Lastly, we would like to acknowledge the use of modern technologies, open-source libraries, and research resources in the fields of Artificial Intelligence, Natural Language Processing, and legal technology, which played a vital role in building and enhancing this system.

This project has been a valuable learning experience, and we are truly grateful to everyone who contributed to its successful completion.

Chandan Boruah (240231025)

Hari om Singh (240231032)

Kunal Rastogi (240231034)

Ritik Kumar (240231050)

MCA 2nd Year

TABLE OF CONTENTS

TITLE **PAGE**

Abstract II
Certificate IV
Acknowledgement V
List of Figures & Tables VIII
CHAPTER 1: INTRODUCTION 1
1.1 Overview 1
1.2 Background of Study 2
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Significance of the Study 2
1.6 Scope and Limitations 3
1.7 Applications of Legal Mind AI 3

CHAPTER 2: LITERATURE REVIEW 4
2.1 Introduction 4
2.2 Research Gap 5
2.3 Contribution of Proposed System 5
CHAPTER 3: WORK METHODOLOGY 6
3.1 System Architecture 6
3.2 RAG Pipeline 7
3.3 AI-Driven Document Summarization 8
3.4 Risk Classification & Analysis 8
3.5 Algorithm Workflow (Hybrid RAG Model) 9
3.6 Algorithm (RAG) 9
3.7 Work Methodology 10
3.8 Tools and Techniques Used 12
CHAPTER 4: DESIGN AND DEVELOPMENT DETAILS 14
4.1 System Architecture Design 14
4.2 Unified Modeling Language (UML) Diagrams 15
4.3 Frontend Development 20
4.4 Backend Development 20

4.5 AI Implementation (RAG) 20
4.6 Database Schema Design (Data Dictionary) 21
4.7 API Implementations 22
4.8 Integration 24
4.9 Testing 24

CHAPTER 5: EXPERIMENTAL RESULTS AND ANALYSIS 25
5.1 Document Analysis Output 25
5.2 Risk Classification Results 26
5.3 Chat with Document Results 26
5.4 User Interface Evaluation 27
5.5 System Performance Analysis 28
5.6 Comparison with Traditional Systems 28
5.7 Observation 29
CHAPTER 6: DISCUSSION 30
6.1 Overview 30
6.2 Advantages of the System 30
6.3 Comparison with Existing Systems 30
6.4 Practical Applications 31
6.5 Challenges Faced 31
6.6 Limitations of the System 31
CHAPTER 7: CONCLUSION 32
7.1 Conclusion 32
7.2 Key Achievements 32
7.3 Final Remarks 33
CHAPTER 8: LIMITATIONS AND FUTURE SCOPE 34
8.1 Limitations 34
8.2 Future Scope 34
8.3 Summary 34
REFERENCES 35
APPENDIX 36
I. System Modules 36
II. Code Snippets 37
III. Output Screens 39

LIST OF FIGURES AND TABLES

- **Figure 3.1:** System Architecture of Legal Mind AI
- **Figure 3.2:** RAG Pipeline Architecture
- **Figure 4.1:** System Workflow of LegalMind AI
- **Figure 4.2:** Use Case Diagram
- **Figure 4.3:** ER Diagram
- **Figure 4.4:** Sequence Diagrams
- **Figure 4.5:** AI Implementation *RAG Diagram*
- **Figure 5.1:** Document Summary Output
- **Figure 5.2:** Risk Classification Categories
- **Figure 5.3:** Chat Interface
- **Figure 5.4:** User Dashboard

- **Table 5.1:** Comparison with Traditional System
- **Table 6.1:** Comparison with Existing System

CHAPTER 1
INTRODUCTION

1.1 Overview

The legal domain is characterized by complex, lengthy, and highly technical documents such as contracts, agreements, policies, and legal notices. These documents are often difficult for common users to interpret without professional assistance. Misinterpretation of legal content can lead to serious consequences, including financial loss, legal disputes, and contractual risks. Therefore, there is a growing need for intelligent systems that can simplify legal information and make it more accessible.

With the rapid advancement of Artificial Intelligence (AI) and Natural Language Processing (NLP), it has become possible to automate the understanding of textual data. AI-driven systems can analyze large volumes of text, identify patterns, and generate meaningful insights. In the legal domain, these technologies can be used to extract important clauses, summarize documents, and identify potential risks.

LegalMind AI is an AI-powered legal document analysis system designed to assist users in understanding legal documents efficiently. The system allows users to upload legal documents and receive structured outputs such as summaries, clause explanations, and risk classification (High, Medium, Low). By leveraging Retrieval-Augmented Generation (RAG), the system ensures that responses are context-aware and grounded in the uploaded document.

In addition to AI-based analysis, LegalMind AI integrates a role-based system that includes Users, Lawyers, and Administrators. While AI provides initial insights, users can also connect with verified lawyers for professional consultation. This hybrid approach combines the speed of AI with the reliability of human expertise.

The system is built using modern technologies including the MERN stack (MongoDB, Express.js, React.js, Node.js) along with a Python-based AI processing layer powered by FastAPI. It utilizes embedding models, vector databases, and Large Language Models (LLMs) to perform semantic search and generate intelligent, context-aware responses.

1.2 Background of Study

Traditionally, legal assistance has been dependent on manual consultation with lawyers, which can be time-consuming and costly. Many individuals lack access to immediate legal guidance due to financial or geographical constraints. Although online legal platforms exist, they often provide static information without personalized insights.

LegalMind AI addresses this gap by introducing an intelligent system capable of providing instant, AI-driven legal analysis while also enabling seamless interaction with professional lawyers. This approach enhances accessibility and improves decision-making for users.

1.3 Problem Statement

Understanding legal documents is a challenging task for non-experts due to complex terminology and lengthy content. Existing systems either rely entirely on human expertise or provide limited automated assistance. There is a need for an intelligent system that can analyze legal documents, provide contextual insights, identify potential risks, and enable interaction with both AI and legal professionals.

1.4 Objectives

- To develop an AI-powered legal document analysis system
- To implement risk classification (High, Medium, Low)
- To enable chat-based interaction with documents
- To design a role-based system (User, Lawyer, Admin)
- To integrate AI with real-world legal consultation

1.5 Significance of the Study

This project contributes to the field of Legal Technology (LegalTech) by making legal knowledge more accessible, reducing dependency on manual document analysis, providing faster and more efficient legal insights, and enhancing decision-making for individuals and businesses.

1.6 Scope and Limitations

Scope:

- Supports analysis of legal documents such as contracts and agreements
- Provides AI-based summarization and risk detection
- Enables user-lawyer interaction through the platform
- Implements scalable web-based architecture

Limitations:

- AI responses may not replace professional legal advice
- Accuracy depends on document quality and context
- Requires internet and API-based AI services

1.7 Applications of LegalMind AI

- Contract Review
- Legal Risk Analysis
- Business Agreement Verification
- Employment Contract Analysis
- Consumer Legal Assistance

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction

The rapid advancement of Artificial Intelligence (AI), Natural Language Processing (NLP), and Legal Technology (LegalTech) has significantly transformed the way legal information is processed and accessed. In recent years, researchers and developers have focused on building intelligent systems capable of analyzing textual data, extracting meaningful insights, and assisting users in decision-making processes.Traditional legal systems rely heavily on manual interpretation of documents by legal professionals. This approach is time-consuming, costly, and not easily accessible to the general public. As a result, there has been a growing interest in developing automated systems that can simplify legal document analysis and improve accessibility.Several studies have explored the use of NLP techniques for document summarization and information extraction. These systems are capable of identifying key clauses, extracting relevant information, and generating summaries. However, many of these approaches lack contextual understanding and often fail to provide accurate insights for complex legal documents.Recent advancements in Large Language Models (LLMs) have improved the ability of AI systems to understand and generate human-like text. Researchers have utilized transformer-based models to develop intelligent assistants capable of answering user queries. While these models show promising results, they often generate generic responses that are not grounded in specific documents.To address this limitation, Retrieval-Augmented Generation (RAG) has emerged as an effective approach. RAG combines semantic retrieval with generative models, allowing systems to retrieve relevant document segments and generate responses based on actual context. This significantly improves accuracy and reduces hallucination in AI-generated outputs.In the legal domain, some platforms provide digital legal assistance and lawyer directories. These systems allow users to search for legal professionals or access predefined legal information. However, they lack integration with AI-driven document analysis and do not offer real-time interaction with uploaded documents.

LegalMind AI builds upon these existing technologies by integrating NLP, RAG, and role-based access into a single unified system. It not only analyzes legal documents but also enables users to

interact with them through a chat interface and connect with verified lawyers when needed.

2.2 Research Gap

- Lack of systems combining AI analysis with real lawyer consultation
- Limited contextual understanding in traditional NLP models
- Absence of real-time interaction with legal documents
- High dependency on manual legal interpretation

2.3 Contribution of Proposed System

- Integration of Retrieval-Augmented Generation (RAG) for accurate responses
- Chat-based interaction with legal documents
- Risk classification (High, Medium, Low)
- Hybrid model of AI + Lawyer consultation
- Scalable and user-friendly architecture

CHAPTER 3
WORK METHODOLOGY

3.1 System Architecture

LegalMind AI follows a multi-layer architecture comprising a frontend, backend, AI processing layer, and databases. This modular design ensures scalability, maintainability, and efficient data processing.

- Frontend: React.js for UI, dashboards, and document upload interface
- Backend: Node.js & Express.js for API handling and business logic
- AI Layer: Python (FastAPI) for NLP processing and RAG execution
- Database: MongoDB for application data and Vector DB (Pinecone) for embeddings

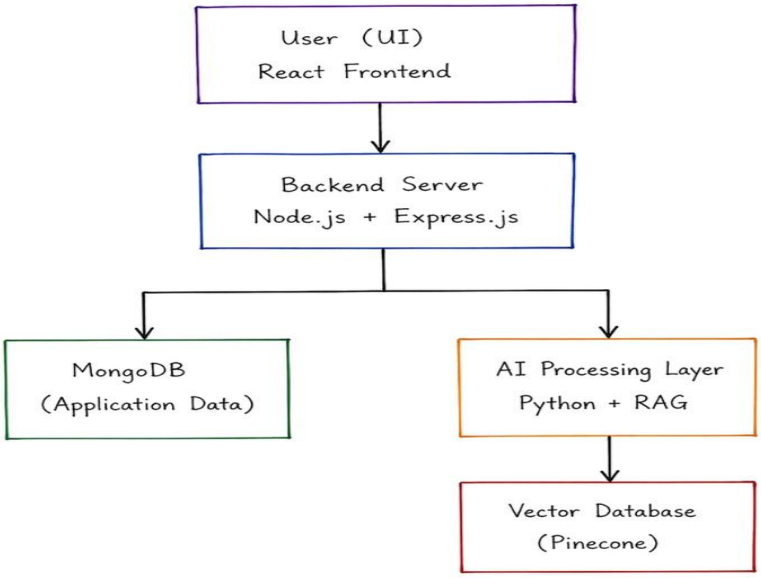


Figure 3.1 System Architecture of Legal Mind AI

3.2 RAG Pipeline

The system uses Retrieval-Augmented Generation (RAG) to ensure context-aware and accurate responses grounded in the uploaded documents.

- **Document Ingestion & Text Extraction:** Utilizing **PyMuPDF** to accurately parse unstructured legal PDFs and extract clean, UTF-8 formatted text while discarding unnecessary visual noise.
- **Semantic Text Segmentation (Chunking):** Since LLMs have token limits, the document is divided into smaller, contextually meaningful segments using **RecursiveCharacterTextSplitter** with defined chunk overlap. This ensures that legal clauses are not abruptly cut off halfway.
- **Dual Embedding Generation:** The system converts text chunks into mathematical arrays. It generates **Dense Embeddings** using the HuggingFace **all-MiniLM-L6-v2** model to capture the contextual meaning, and **Sparse Embeddings (BM25)** using **pinecone-text** to capture exact keyword importance.
- **Vector Storage:** Both dense and sparse vectors, along with document metadata, are upserted into a Pinecone Vector Database using the **dotproduct** metric.
- **Hybrid Retrieval (Query Processing):** When a user asks a query, it undergoes the same dual-embedding process. The system retrieves the top relevant chunks by merging dense semantic similarity and sparse keyword matching scores.
- **Prompt Engineering & Strict Grounding:** The retrieved context and the user query are injected into a highly restricted Prompt Template. This explicitly instructs the LLM to refuse answering if the answer is not present in the context, effectively eliminating AI hallucination.
- **LLM Response Generation:** The aggregated context is sent to the high-speed Groq LLaMA-3 model, which streams back a natural, conversational, and legally sound answer.
- **Hybrid Retrieval Mechanism:** Combines dense vector search for conceptual meaning with sparse BM25 search for exact legal keyword matching using **pinecone-text.**
- **Contextual Prompting:** Strictly grounds the LLM to refuse answering if the retrieved context is insufficient, preventing AI hallucination.
- LLM-based Response Generation

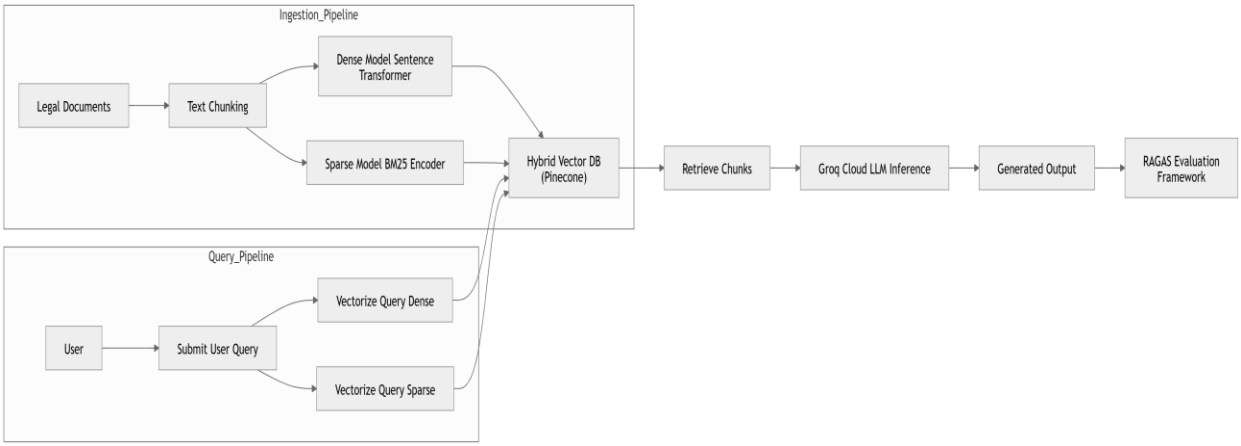


Figure 3.2: RAG Pipeline Architecture

3.3 AI-Driven Document Summarization

Reading lengthy legal documents is tedious for non-experts. The system incorporates an automated summarization module alongside the RAG pipeline.

- **Text Aggregation:** Upon document upload, the AI engine processes the first highly critical segments of the document (or loops over chunks) to capture the preamble, parties involved, and primary objective of the contract.
- **Information Distillation:** The extracted content is passed to the LLM with a specialized instruction prompt to synthesize a concise, non-technical overview.
- **Output Generation:** The system returns a digestible summary that helps users immediately understand the core intent of the document without reading dozens of pages.

3.4 Risk Classification & Analysis

The AI engine performs an automated risk assessment to protect users from hidden legal traps. It analyzes the retrieved clauses using pattern recognition and predefined legal heuristics.

- **High Risk (Red Flag):** Detects severe liabilities, immediate financial penalties, strict termination clauses, and highly biased obligations that strongly favor the opposing party.
- **Medium Risk (Yellow Flag):** Identifies ambiguous terms, vague conditional clauses, or statements that lack clear legal definition and might lead to future disputes.
- **Low Risk (Green Flag):** Highlights standard boilerplate clauses, informational content, and balanced terms that pose no immediate threat to the user.

3.5 Algorithm Workflow (Hybrid RAG Model)

Input: Legal Document D, User Query Q **Output:** Context-Grounded Answer A

1. Extract raw text T from Document D.
2. Split T into semantically overlapping chunks [C1, C2, ..., Cn].
3. For each chunk Ci: a. Generate Dense Vector Vd(Ci) using Sentence-Transformers. b. Generate Sparse Vector Vs(Ci) using BM25 Encoder. c. Upsert {Vd(Ci), Vs(Ci), Metadata} into Pinecone Database DB.
4. Convert User Query Q into Query Dense Vector Vd(Q) and Query Sparse Vector Vs(Q).
5. Perform top-K Hybrid Similarity Search in DB: Score = (Dense_Sim * alpha) + (Sparse_Sim * (1-alpha))
6. Retrieve top relevant chunks [C_rel].
7. Construct Prompt P by concatenating [C_rel] and Q.
8. Pass P to Groq LLM inference layer.
9. If Q is unanswerable from [C_rel], Return "Context Insufficient".
10. Else, Return Generated Answer A.

3.6 Algorithm (RAG)

Input: Document D, Query Q

- Extract text from document
- Divide text into chunks

- Generate embeddings for each chunk
- Store embeddings in vector database
- Convert query into embedding
- Perform Hybrid Search computing the dotproduct of both Dense Embeddings and Sparse (BM25) Embeddings.
- Re-rank retrieved chunks based on cumulative hybrid similarity scores.
- Pass context to LLM
- Generate final response

3.7 Work Methodology

The development of the LegalMind AI platform followed a structured, iterative, and industry-oriented methodology by integrating modern web technologies with advanced Artificial Intelligence techniques. The approach ensured scalability, reliability, maintainability, and high accuracy of the system

1. Requirement Analysis

    - Functional Requirements: Secure document upload, AI-based risk analysis, chatbot interaction, and lawyer dashboard.
    - Non-Functional Requirements: Data security, system performance, scalability, and role-based access control (RBAC).
    - Stakeholders: General users, legal professionals, and administrators.

2. System Design

    - UML Diagrams: Use Case Diagram, ER Diagram, and Sequence Diagram were created.
    - Architecture Layers: Frontend (React), Backend (Node.js), AI Layer (Python), Database (MongoDB).
    - Clear separation of concerns ensured maintainability and extensibility.

3. Technology Selection

    - MERN Stack (MongoDB, Express.js, React.js, Node.js) for web development.

- Python for AI and NLP processing.
- RAG (Retrieval-Augmented Generation) pipeline for accurate and context-based responses.

4. Data Processing

- Text Extraction from PDFs and DOCX files.
- Chunking of large documents into smaller segments.
- Embedding Generation for semantic understanding.
- Vector Database for fast retrieval.

5. Implementation

- User Module: Upload documents, view analysis, interact with chatbot.
- Lawyer Module: Manage consultations and review cases.
- Admin Module: Manage users and verify lawyers.
- REST APIs for frontend-backend communication.

6. Testing

- Unit Testing: Testing individual components.
- Integration Testing: Testing interaction between modules.
- System Testing: End-to-end workflow validation.
- API Testing: Using Postman for endpoint validation.
- AI Model Testing: Evaluating RAG accuracy.
- Performance Testing: Checking load and response time.
- UI/UX Testing: Cross-browser compatibility.
- Security Testing: Authentication and data protection.

7. Deployment

- Cloud Deployment for scalability.
- Accessible via web browsers.
- CI/CD for continuous updates.

3.8 Tools and Techniques Used

The **tools** and **technologies** used in the LegalMind AI project, categorized by their purpose in the system:

1. Programming Languages
   - **JavaScript / Node.js**: Used for the frontend application and the primary backend API gateway.
   - **Python**: Used for the AI engine, document processing, and NLP tasks.

2. Frontend Technologies (Web App)
   - **React.js**: Core UI library for building the user interfaces.
   - **Vite**: Ultra-fast build tool and development server.
   - **Tailwind CSS**: Utility-first CSS framework for styling.
   - **React Router DOM**: For handling page navigation and routing.
   - **Socket.io-Client**: For real-time updates and chat communication.
   - **Axios**: For making HTTP requests to the backend.
   - **Lucide-React**: For scalable SVG icons.
3. Backend Technologies (Node.js API)
   - **Express.js**: Web framework for building the REST APIs.
   - **MongoDB & Mongoose**: NoSQL database and Object Data Modeling (ODM) library.
   - **JSON Web Tokens (JWT) & Bcrypt.js**: For secure authentication, session management, and password hashing.
   - **Socket.io**: WebSockets for real-time, bidirectional communication.
   - **Multer**: Middleware for handling multipart/form-data (file uploads).
   - **Cloudinary**: Cloud service for managing and storing uploaded files.

4. AI Engine & Data Processing (Python)
   - **FastAPI**: High-performance web framework for the Python backend.
   - **LangChain & LangGraph**: Frameworks for orchestrating Large Language Models (LLMs) and building the RAG pipeline.
   - **Groq**: High-speed LLM inference (integrated via langchain-groq).

- **Pinecone**: Vector database for storing and querying document embeddings.
- **PyMuPDF**: For parsing and extracting raw text from uploaded PDF documents.
- **Pydantic**: For strict data validation and settings management.
- **Sentence-Transformers**: Offline HuggingFace embeddings (all-MiniLM-L6-v2) used for generating secure, local text vectors.
- **Pinecone-Text (BM25)**: Utilized for generating sparse vectors to enable exact keyword/syntactical matching in Hybrid Search.
- **RAGAS (Retrieval Augmented Generation Assessment)**: Employed for automated, quantitative evaluation of the AI pipeline's accuracy and grounding.

5. Development & Testing Tools

- **Git**: For version control (indicated by the .git directory).
- **Postman**: Used for rigorous API endpoint testing (indicated by the Postman collections in your backend/postman folder).
- **Nodemon**: For automatically restarting the Node.js backend during development.

CHAPTER 4
DESIGN AND DEVELOPMENT DETAILS

4.1 System Design

- Modular multi-layer architecture (Frontend, Backend, AI, Data).
- Frontend (React): UI, upload, dashboards, chat.
- Backend (Node + Express): APIs, auth, logic.
- AI (Python + FastAPI): NLP, RAG, LLM responses.
- Data: MongoDB + Vector DB (FAISS/Pinecone).
- Workflow: Upload → Backend → AI → Vector DB → Response.

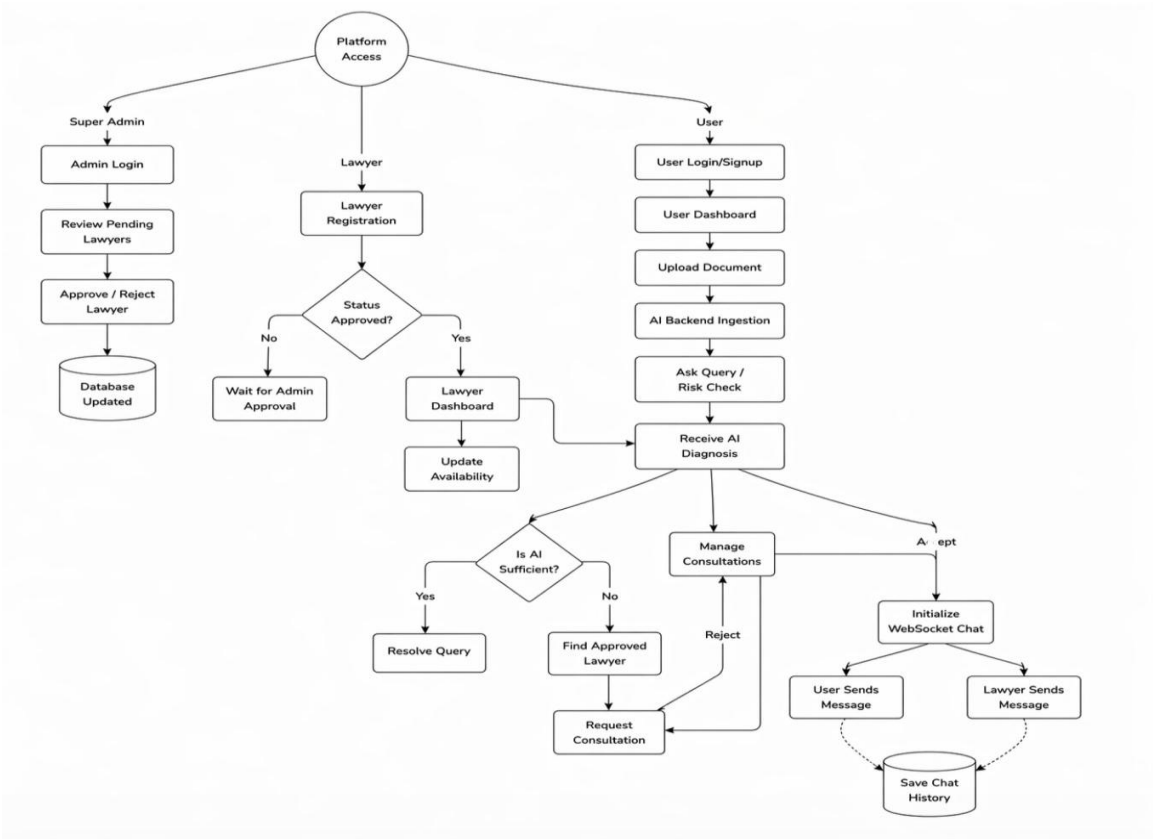


Figure 4.1: System Workflow of LegalMind AI

4.2 Unified Modeling Language (UML) Diagrams

To visualized the architecture and dynamic behavior of Legal Mind Term & condition AI, Standard UML diagrams were constructed .

4.2.1 Use Case Diagram

The Use Case Diagram of the LegalMind AI platform represents the interaction between different actors—User, Lawyer, and Super Admin—and the system. The user can upload legal documents, analyze risks, and interact with the system through AI-based chat.

The core functionality is handled by the RAG (Retrieval-Augmented Generation) engine, which processes documents, generates embeddings, and provides context-aware responses.

Lawyers can manage consultation requests and communicate with users, while the Super Admin is responsible for approving lawyer registrations.

Overall, the diagram illustrates a role-based system that integrates AI capabilities with human legal expertise to provide efficient and intelligent legal assistance.

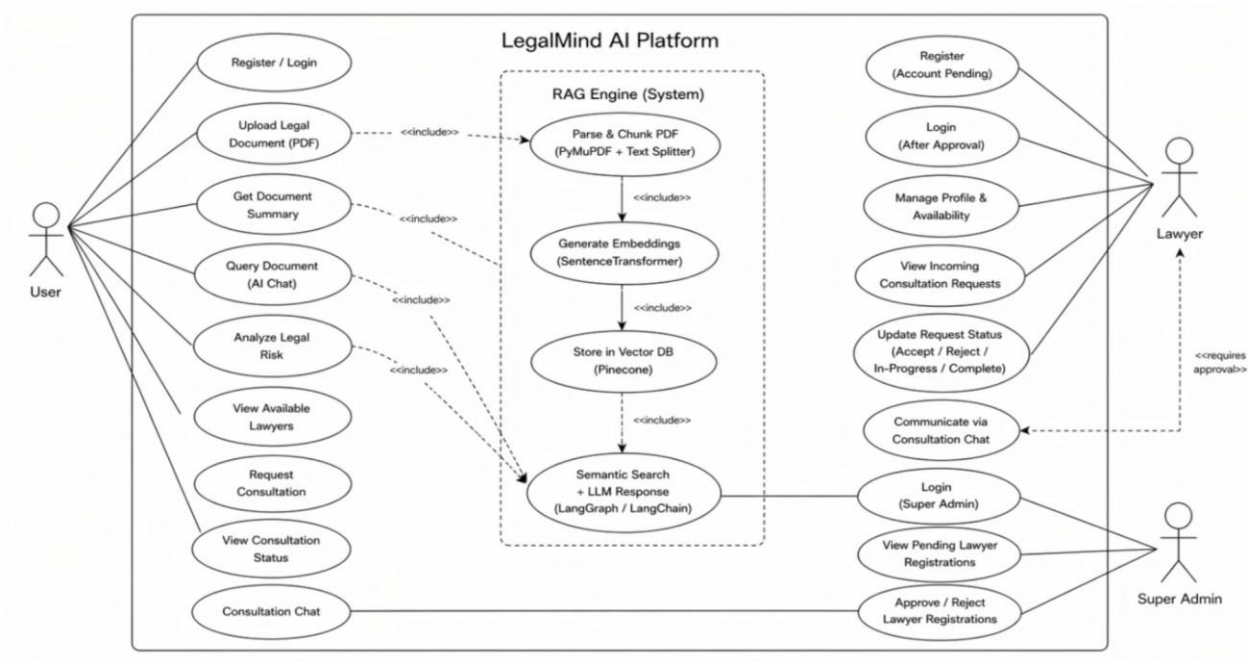


Figure 4.2: Use Case Diagram

4.2.2 ER Diagram

The ER (Entity-Relationship) diagram represents the database design of the LegalMind AI system and illustrates how different entities are connected to manage legal data efficiently. The main entities include **User, Lawyer, Document, Consultation Request, and Consultation Message**.

The **User** entity stores user details and has a one-to-many relationship with the **Document** entity, as a user can upload multiple legal documents. Each document contains information such as file details and status.

The **Consultation Request** entity acts as a link between users and lawyers. A user can create multiple consultation requests, and a lawyer can handle multiple requests, forming one-to-many relationships on both sides.

The **Consultation Message** entity represents communication between user and lawyer. Each consultation request can contain multiple messages, enabling continuous interaction.

Overall, the ER diagram defines clear relationships and ensures efficient data organization, smooth communication, and effective handling of legal documents within the system.

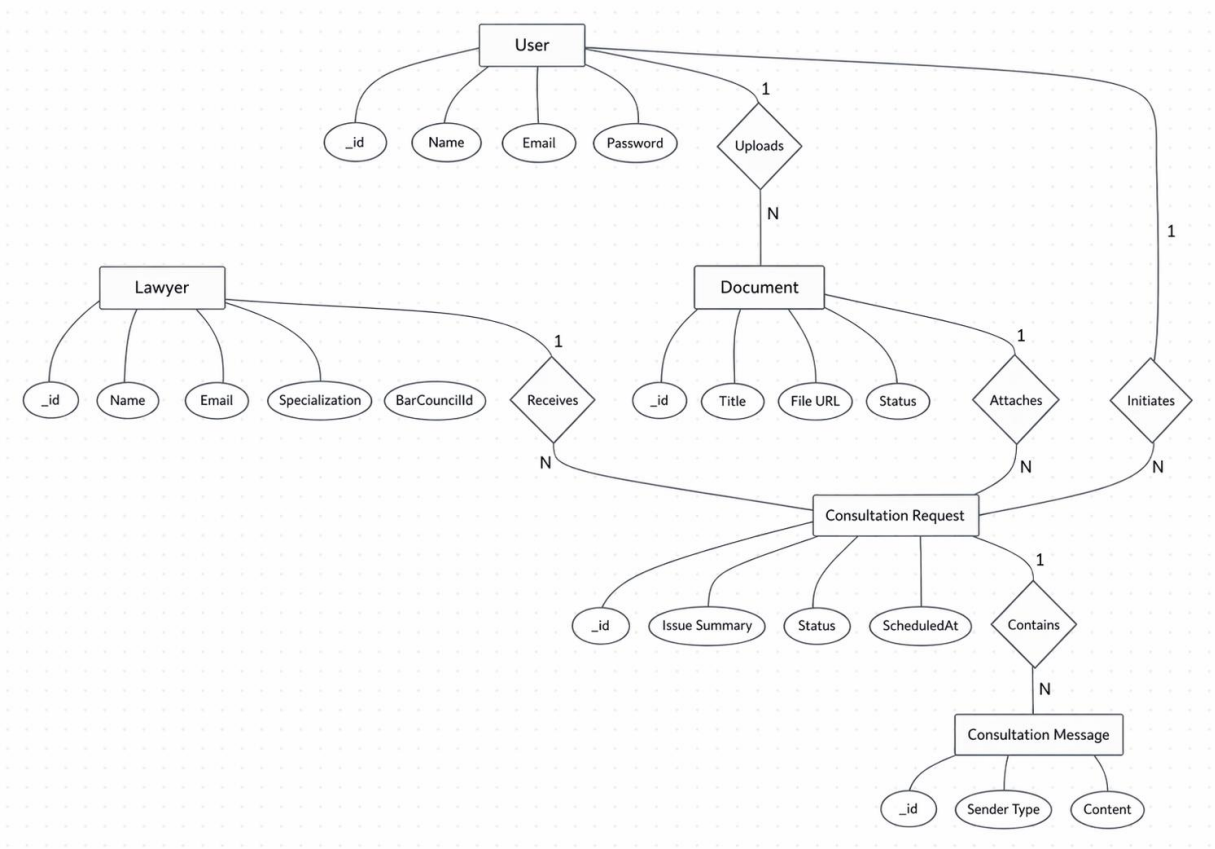


Figure 4.3 : ER Diagram

4.2.3 Sequence Diagram

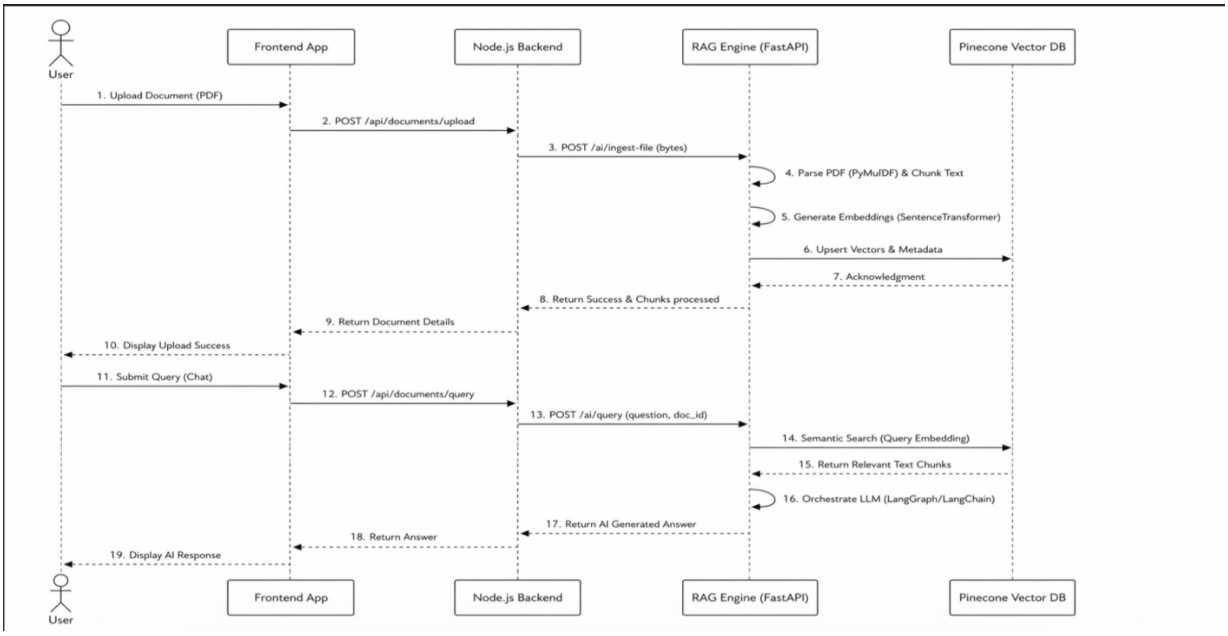
I. AI Document Processing & Querying (RAG Flow)

This sequence describes how a User uploads a PDF, how the system ingests it using the RAG Engine, and how the User later queries that document for AI-generated answers.

Workflow Context

1. The User uploads a PDF via the frontend.
2. The Backend passes the file to the Python RAG Engine.
3. The RAG Engine extracts text, chunks it, embeds it, and stores it in Pinecone.
4. The User asks a question about the document.

5. The RAG Engine retrieves relevant chunks from Pinecone, orchestrates the LLM response via LangGraph, and returns the answer.

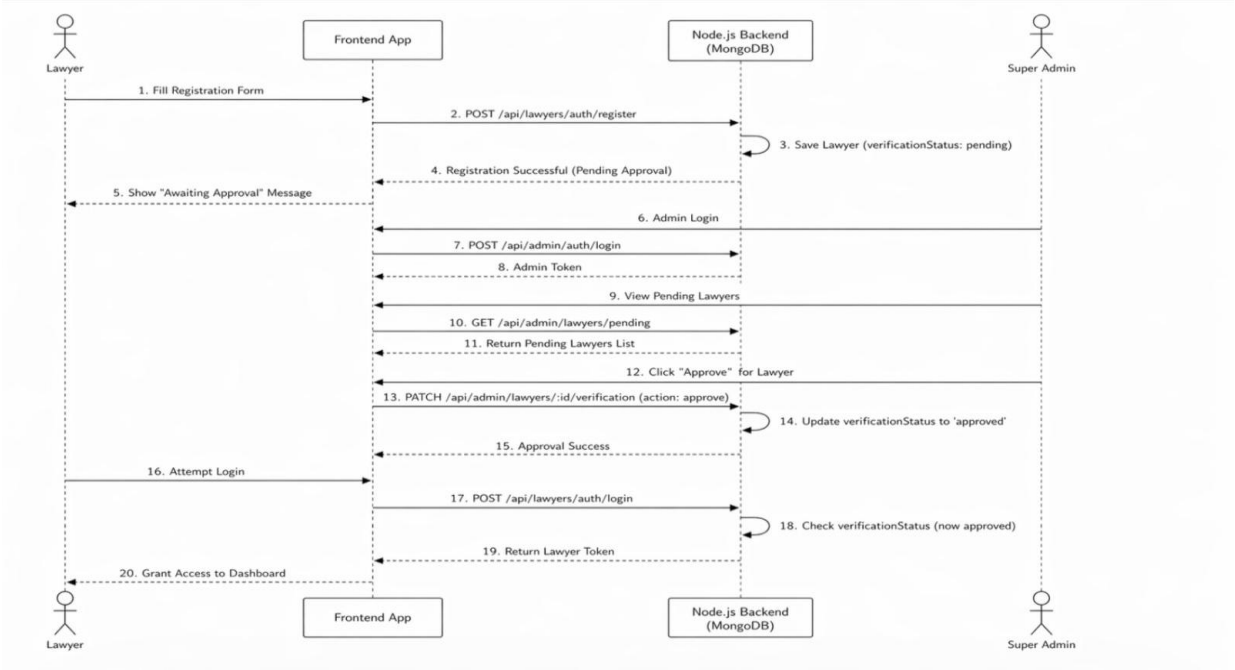


II. Lawyer Consultation Flow

This sequence models the process of a User requesting a human lawyer when AI is insufficient, and the Lawyer accepting the request to begin a chat.

Workflow Context

1. The User fetches the list of available, approved lawyers.
2. The User submits a consultation request (including a summary of their issue).
3. The Lawyer views their dashboard and accepts the pending request.
4. The User and Lawyer exchange messages in the Consultation Chat.



III. Lawyer Onboarding & Super Admin Approval Flow

This sequence shows the administrative process required to ensure only verified legal professionals can offer services on the platform.

Workflow Context

1. A Lawyer registers an account. Their initial status is automatically set to pending.
2. The Super Admin logs in, reviews the list of pending lawyers, and explicitly approves the registration.

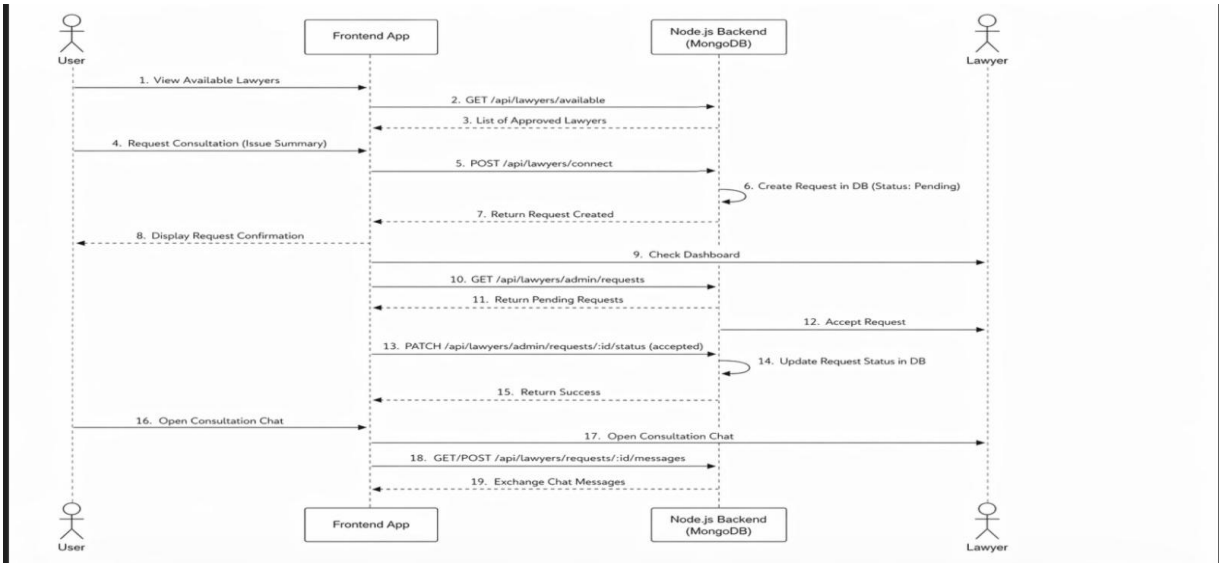


Figure 4.4 : Sequence Diagrams

4.3 Frontend Development

- React-based responsive UI.
- Features: Login, Upload, Dashboard, Chat.
- Uses REST APIs via Axios/Fetch.

4.4 Backend Development

- Node.js + Express backend.
- JWT Authentication.
- Role-based access control.
- Middleware for validation.

4.5 AI Implementation (RAG)

- Python FastAPI AI layer.
- Steps: Extraction → Chunking → Embedding → Retrieval → LLM.

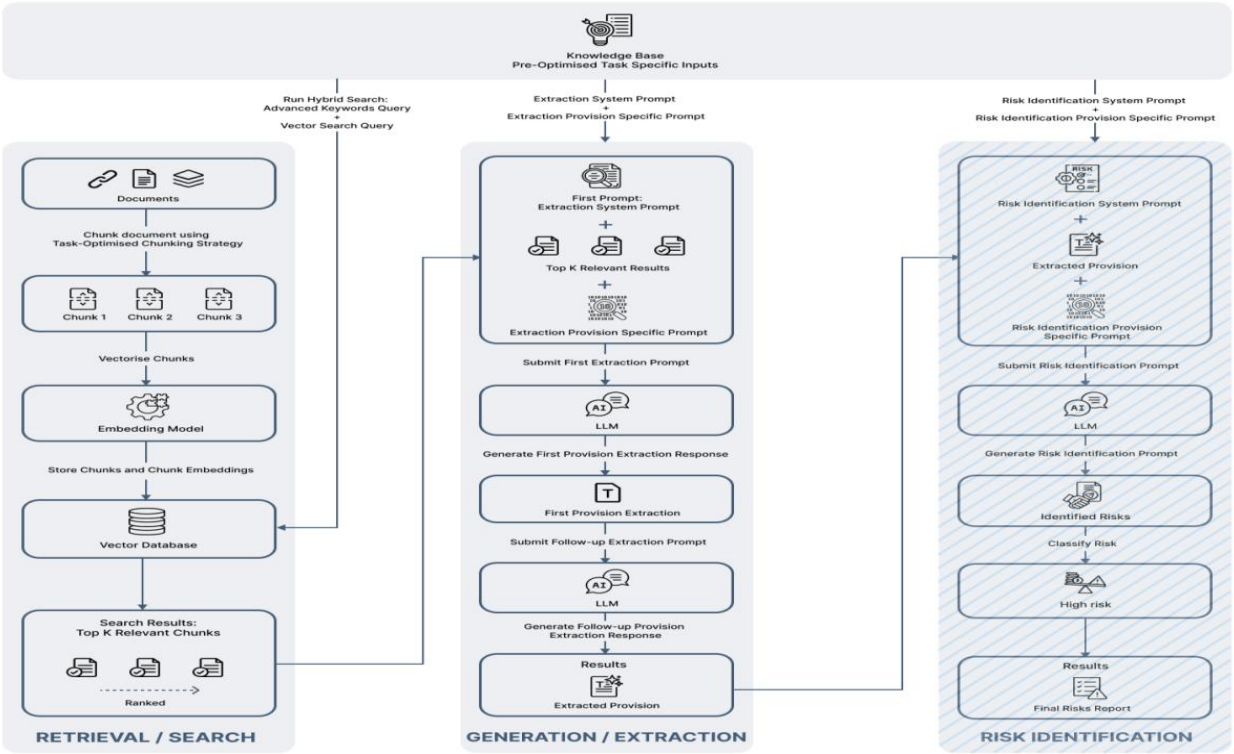


Figure 4.5 : AI Implementation *RAG Diagram*

4.6 Database Design

- MongoDB collections: User, Lawyer, Document, Consultation.
- Vector DB for embeddings.
- DB Schema

User Collection

- id
- name
- email
- password
- role

Lawyer Collection

- id
- name
- specialization
- verificationStatus

Consultation Collection

- id
- userId
- lawyerId
- issueSummary
- status

4.7 API Implementation

This document outlines all the available API endpoints across both the Node.js API Gateway (Backend) and the Python FastAPI (AI Engine).

1. Node.js API Gateway

1.1 Authentication Routes

- POST /register : Registers a new standard user
- POST /login : Authenticates user and returns JWT token

1.2 Document Routes

- GET / : Retrieve uploaded documents (Auth required)
- POST /upload : Upload PDF and trigger AI ingestion
- POST /query : Ask questions to AI chatbot
- POST /analyze-risk : Analyze document risks
- POST /summary : Generate document summary

1.3 Lawyer Routes

1.3.1 Public / Authentication

- POST /auth/register : Register lawyer (pending status)
- POST /auth/login : Login approved lawyer
- GET /auth/status : Check verification status
- GET /available : List available lawyers

1.3.2 User Facing

- POST /connect : Create consultation request
- GET /my-requests : View user requests
- GET /requests/{id}/messages : Chat history

1.3.3 Lawyer Dashboard

- GET /admin/profile : Get lawyer profile
- PUT /admin/profile : Update profile
- PATCH /admin/availability : Update availability
- GET /admin/requests : Get assigned requests
- GET /admin/requests/{id}/messages : Chat history
- PATCH /admin/requests/{id}/status : Update request status
- GET /admin/dashboard/stats : Dashboard statistics

1.4 Super Admin Routes

- POST /auth/login : Admin login
- GET /lawyers/pending : View pending lawyers
- GET /lawyers : View all lawyers
- PATCH /lawyers/{id}/verification : Verify lawyers

2. Python AI Engine (FastAPI)

2.1 Ingestion Endpoints

- POST /ai/ingest : Process file via URL
- POST /ai/ingest-file : Process uploaded file

2.2 AI Processing Endpoints

- POST /ai/query : Answer user queries

- POST /ai/analyze-risk : Detect legal risks
- POST /ai/summary : Generate summary

4.8 Integration

The LegalMind AI system follows a seamless multi-layer integration architecture to ensure smooth data flow and real-time processing. The overall integration pipeline can be represented as: Frontend → Backend → AI Engine → Database → Frontend

- Frontend (React.js): User interacts through web interface for upload, chat, and analysis.
- Backend (Node.js / Express): Handles authentication, validation, and routes requests.
- AI Engine (Python FastAPI with RAG): Processes analysis, queries, and summaries.
- Database (MongoDB + Vector DB): Stores metadata and embeddings for semantic search.
- Response Flow: Results are sent back to frontend via backend.

4.9 Testing

- Unit Testing: Testing individual components and functions.
- Integration Testing: Testing interaction between modules.
- API Testing: Validating endpoints using tools like Postman.
- System Testing: End-to-end workflow validation.
- UI/UX Testing: Checking responsiveness and usability.
- Performance Testing: Evaluating load handling and response time.
- Security Testing: Ensuring authentication and data protection.
- AI Model Testing: Validating accuracy of AI responses.
- AI Model Automated Testing (RAGAS): Evaluated the retrieval and generation pipeline using the RAGAS framework on a custom legal test-set (RAG_Legal_Test_Set.json).
- Faithfulness Metric Check**:** Validated that the AI-generated responses are strictly grounded in the uploaded context and do not hallucinate external facts.
- Answer Relevancy Metric Check: Measured how effectively the AI answers the user's explicit query without providing generic or unrelated information.

CHAPTER 5
EXPERIMENTAL RESULTS AND ANALYSIS

5.1 Document Analysis Output

The system processes uploaded legal documents and generates structured outputs including summaries, clause extraction, and risk classification.

- Document Summary
- Key Clause Extraction
- Risk Classification (High/Medium/Low)
- Contextual Explanation

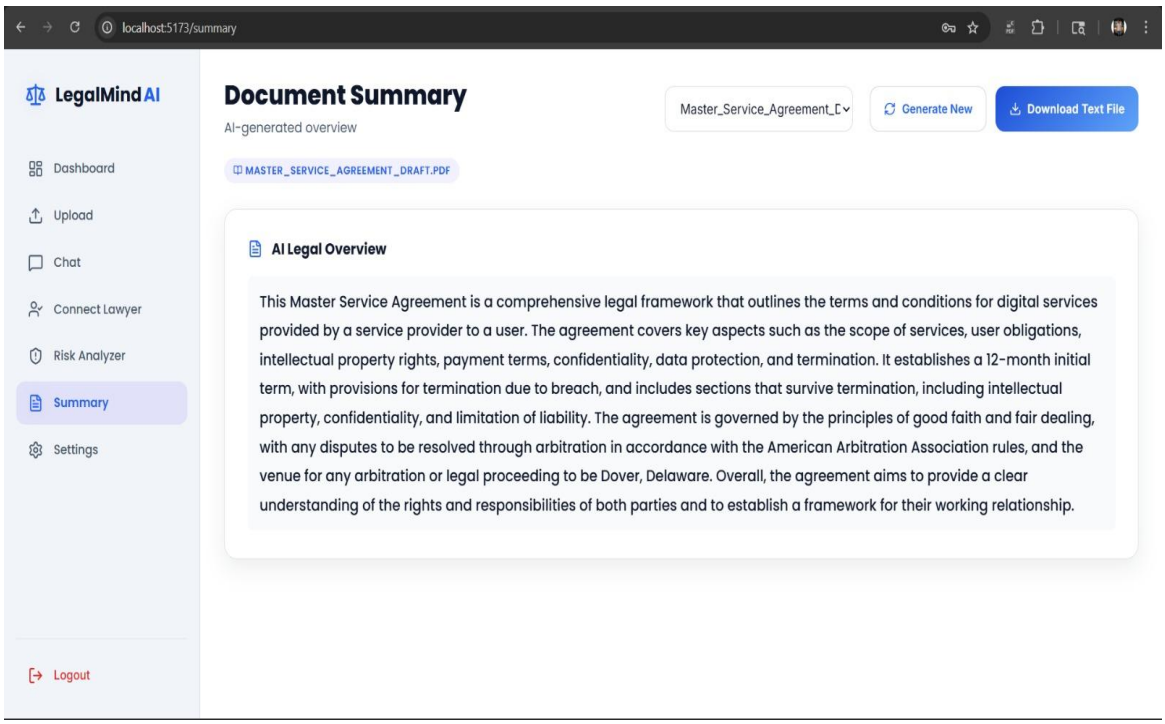


Figure 5.1: Document Summary Output

5.2 Risk Classification Results

Legal content is categorized into High, Medium, and Low risk based on contextual NLP analysis.

- High Risk: Legal liabilities and penalties
- Medium Risk: Ambiguous clauses
- Low Risk: Informational content

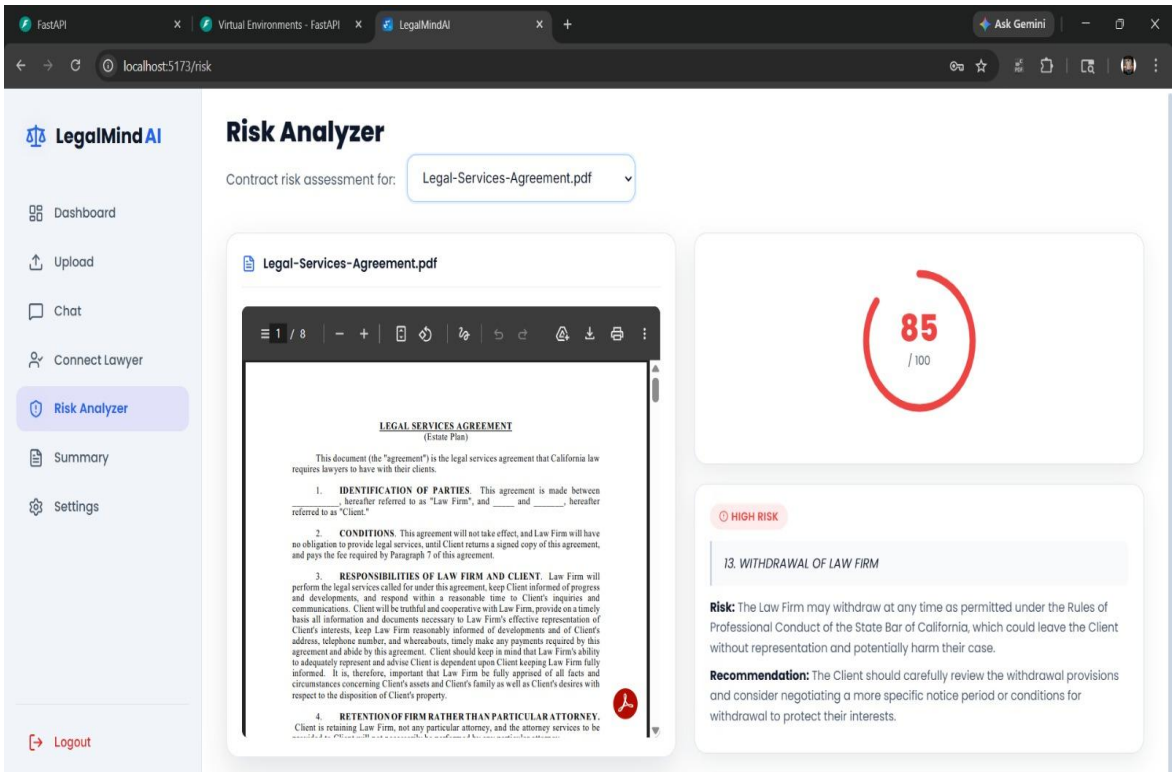


Table 5.2: Risk Classification Categories

5.3 Chat with Document Results

Users can interact with documents via chat to extract insights.

- Ask questions about document
- Get contextual responses
- Retrieve relevant clauses

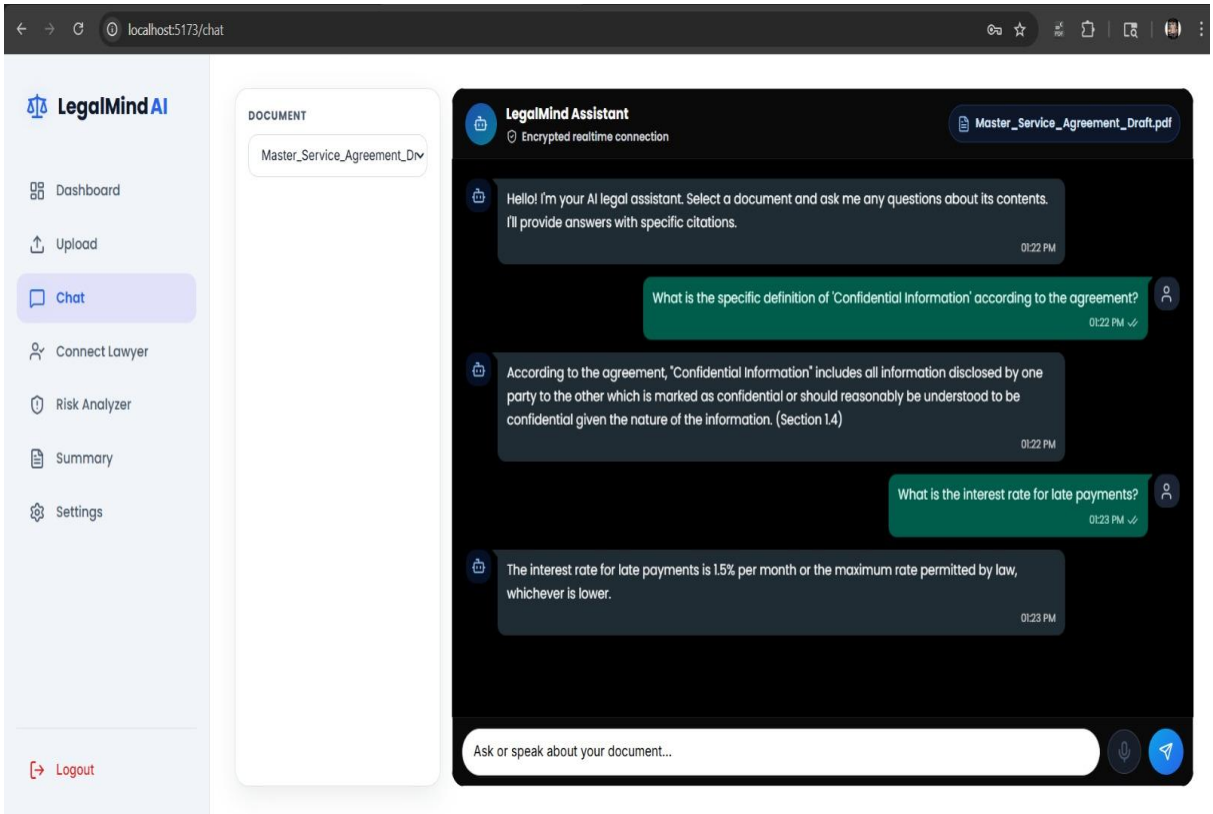


Figure 5.3: Chat Interface

5.4 User Interface Evaluation

The UI is designed for ease of use and accessibility.

- Upload panel
- Analysis dashboard
- Chat interface
- Risk visualization

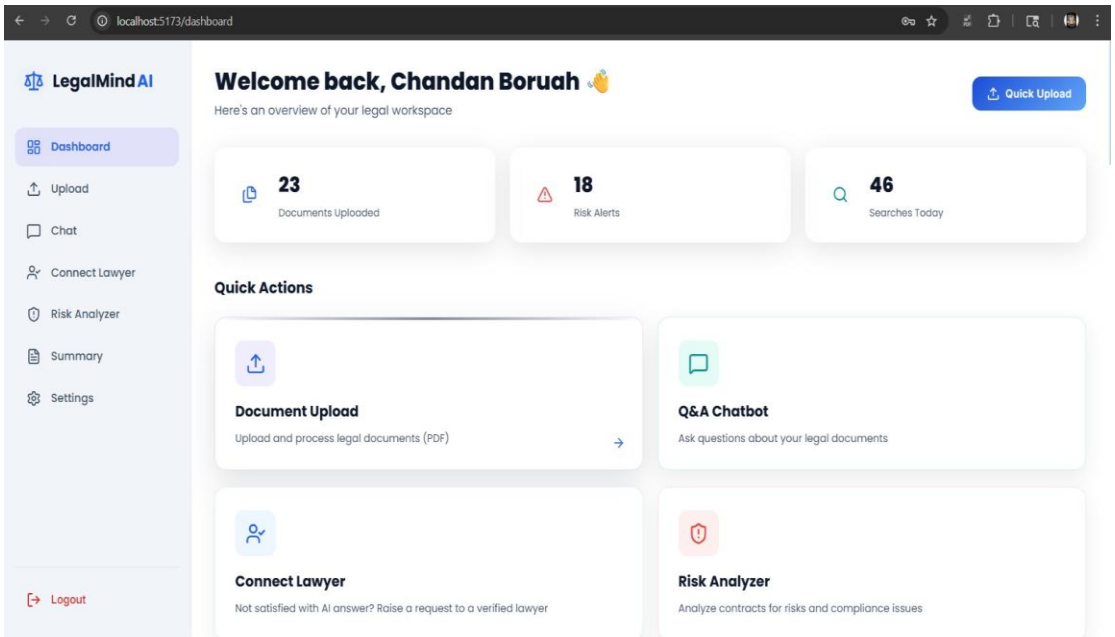


Figure 5.4: User Dashboard

5.5 System Performance Analysis

- High accuracy using RAG
- Response time: 2–5 seconds
- Scalable architecture
- User-friendly interface

5.6 Comparison with Traditional Systems

Feature	Traditional	LegalMind AI
Speed	Slow	Fast
Accuracy	Medium	High
Accessibility	Limited	Easy
Automation	Low	High

Table 5.1 : Comparison with Traditional System

5.7 Observation

- Reduces manual effort
- Provides instant insights
- Combines AI with lawyer expertise
- Useful for individuals and businesses

CHAPTER 6
DISCUSSION

6.1 Overview

This chapter discusses the outcomes of the LegalMind AI system and evaluates its effectiveness in solving the problem of legal document analysis. The system integrates Artificial Intelligence with legal workflows to provide fast, accurate, and user-friendly solutions.

6.2 Advantages of the System

- Automates legal document analysis
- Provides instant insights using AI
- Reduces dependency on manual reading
- Improves accessibility for non-experts
- Supports real-time interaction with documents

6.3 Comparison with Existing Systems

Traditional legal systems rely on manual consultation and static information sources. In contrast, LegalMind AI provides dynamic, AI-driven insights with contextual understanding.

Feature	**Traditional System**	**LegalMind AI**
Analysis Speed	Slow	Fast
Accuracy	Moderate	High
Accessibility	Limited	Easy
Automation	Low	High
User Interaction	Minimal	Interactive Chat

Table 6.1: Comparison with Existing System

6.4 Practical Applications

- Contract and agreement analysis
- Legal risk detection
- Business compliance verification
- Employment contract review
- Consumer rights assistance

6.5 Challenges Faced

- Handling complex legal language
- Ensuring accuracy of AI-generated responses
- Integration of AI with backend systems
- Managing large document sizes

6.6 Limitations of the System

- AI responses may not replace legal professionals
- Dependent on quality of input documents
- Requires internet connectivity
- Possible limitations in edge legal cases

CHAPTER 7
CONCLUSION

7.1 Conclusion

LegalMind AI is an advanced AI-powered system designed to simplify the analysis of complex legal documents. The system successfully integrates modern technologies such as Natural Language Processing (NLP), Retrieval-Augmented Generation (RAG), and the MERN stack to provide efficient, accurate, and user-friendly solutions for legal document understanding.

The project demonstrates that AI can significantly reduce the time and effort required to interpret legal documents by providing structured outputs such as summaries, clause extraction, and risk classification. The chat-based interaction feature further enhances usability by allowing users to query documents and receive context-aware responses.

In addition to automation, the system incorporates a role-based architecture that connects users with verified lawyers, ensuring that professional guidance is available when required. This hybrid approach combines the speed of AI with the reliability of human expertise.

Overall, the system achieves its objectives by improving accessibility, efficiency, and accuracy in legal document analysis. It serves as a practical example of how AI can be applied in the legal domain to assist individuals and organizations in making informed decisions.

7.2 Key Achievements

The major achievements of the LegalMind AI system include:

- Development of an AI-based legal document analysis system
- Implementation of Retrieval-Augmented Generation (RAG)
- Successful integration of MERN stack with Python AI services
- Implementation of risk classification (High/Medium/Low)
- Development of chat-based interaction with documents

- Creation of role-based modules (User, Lawyer, Admin)

7.3 Final Remarks

LegalMind AI highlights the potential of Artificial Intelligence in transforming traditional legal processes. While the system provides strong foundational capabilities, it also opens opportunities for further advancements in Legal Technology. The project reflects both technical implementation and real-world applicability, making it a valuable contribution to the field of AI-driven legal solutions.

CHAPTER 8
LIMITATIONS AND FUTURE SCOPE

8.1 Limitations

While LegalMind AI provides effective legal document analysis, certain limitations exist due to the nature of AI systems and project scope.

- AI-generated responses may not fully replace professional legal advice
- Accuracy depends on the quality and clarity of uploaded documents
- Complex or rare legal cases may not be fully interpreted
- System requires internet connectivity for AI processing
- Performance may vary with very large or unstructured documents

8.2 Future Scope

The system can be further enhanced by incorporating advanced features and improvements:

- Multilingual support for legal documents
- Voice-based interaction with the system
- Integration with legal databases and court records
- Real-time lawyer consultation (video/audio)
- Advanced AI models for improved accuracy
- Mobile application development
- Automated legal report generation

8.3 Summary

LegalMind AI has strong potential for future development. By addressing current limitations and incorporating advanced features, the system can evolve into a comprehensive legal assistant platform, improving accessibility and efficiency in the legal domain

REFERENCES

1. FastAPI, “FastAPI Documentation.” Available: https://fastapi.tiangolo.com/

2. Uvicorn, “Uvicorn Documentation.” Available: https://www.uvicorn.org/

3. Pydantic, “Pydantic Documentation.” Available: https://docs.pydantic.dev/

4. LangChain, “LangGraph Documentation.” Available: https://langchain-ai.github.io/langgraph/

5. LangChain, “LangChain Documentation.” Available: https://python.langchain.com/

6. Groq, “Groq API Documentation.” Available: https://console.groq.com/docs

7. Pinecone, “Pinecone Documentation.” Available: https://docs.pinecone.io/

8. N. Reimers and I. Gurevych, “Sentence-BERT,” EMNLP-IJCNLP, 2019.

9. Hugging Face, “all-MiniLM-L6-v2.” Available:
https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

10. PyMuPDF, “Documentation.” Available: https://pymupdf.readthedocs.io/

11. Express.js, “Documentation.” Available: https://expressjs.com/

12. Socket.IO, “Documentation.” Available: https://socket.io/docs/v4/

13. Mongoose, “Documentation.” Available: https://mongoosejs.com/docs/

14. Cloudinary, “Documentation.” Available: https://cloudinary.com/documentation

15. React, “Documentation.” Available: https://react.dev/

16. React Router, “Documentation.” Available: https://reactrouter.com/

17. Axios, “Documentation.” Available: https://axios-http.com/docs/intro

18. Tailwind CSS, “Documentation.” Available: https://tailwindcss.com/docs

19. Vite, “Documentation.” Available: https://vite.dev/guide/

20. M. Jones et al., “JSON Web Token (JWT),” RFC 7519, 2015

Appendix

I . System Modules

User Module

- User Registration & Login
- Upload Legal Document
- View Analysis & Risk Classification
- Chat with Document
- Request Lawyer Consultation

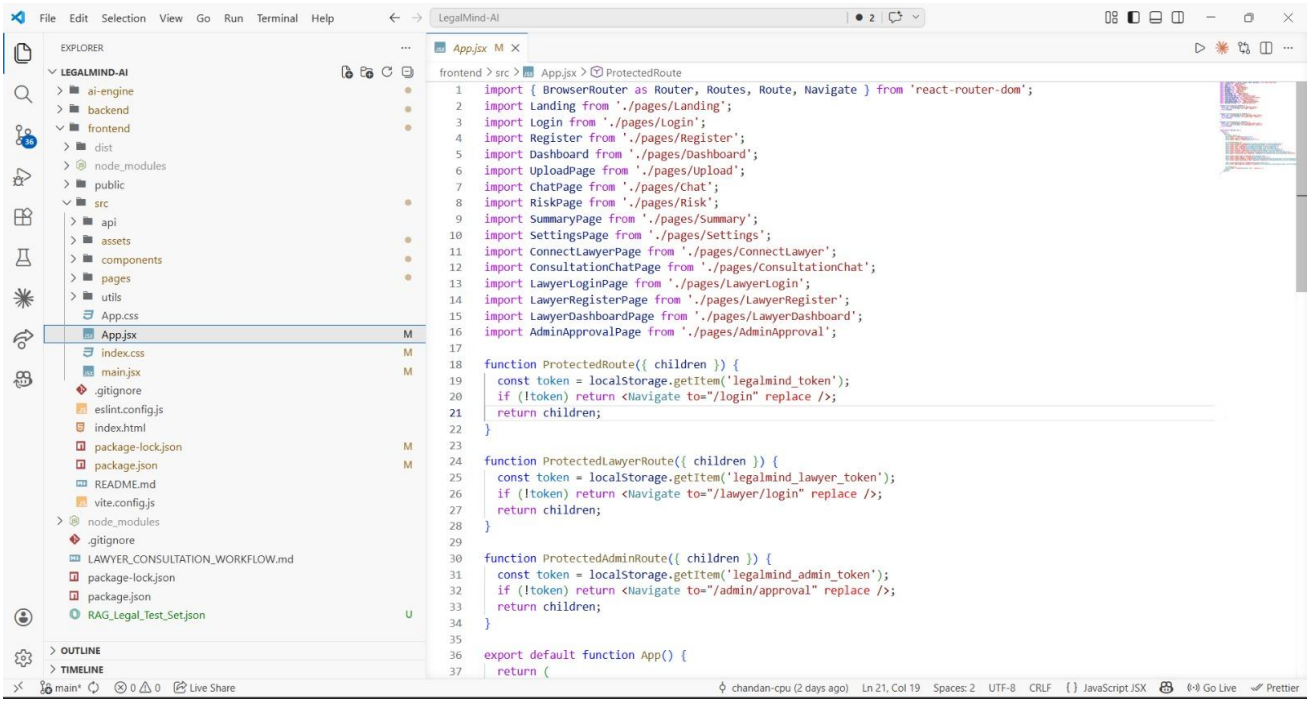
Lawyer Module

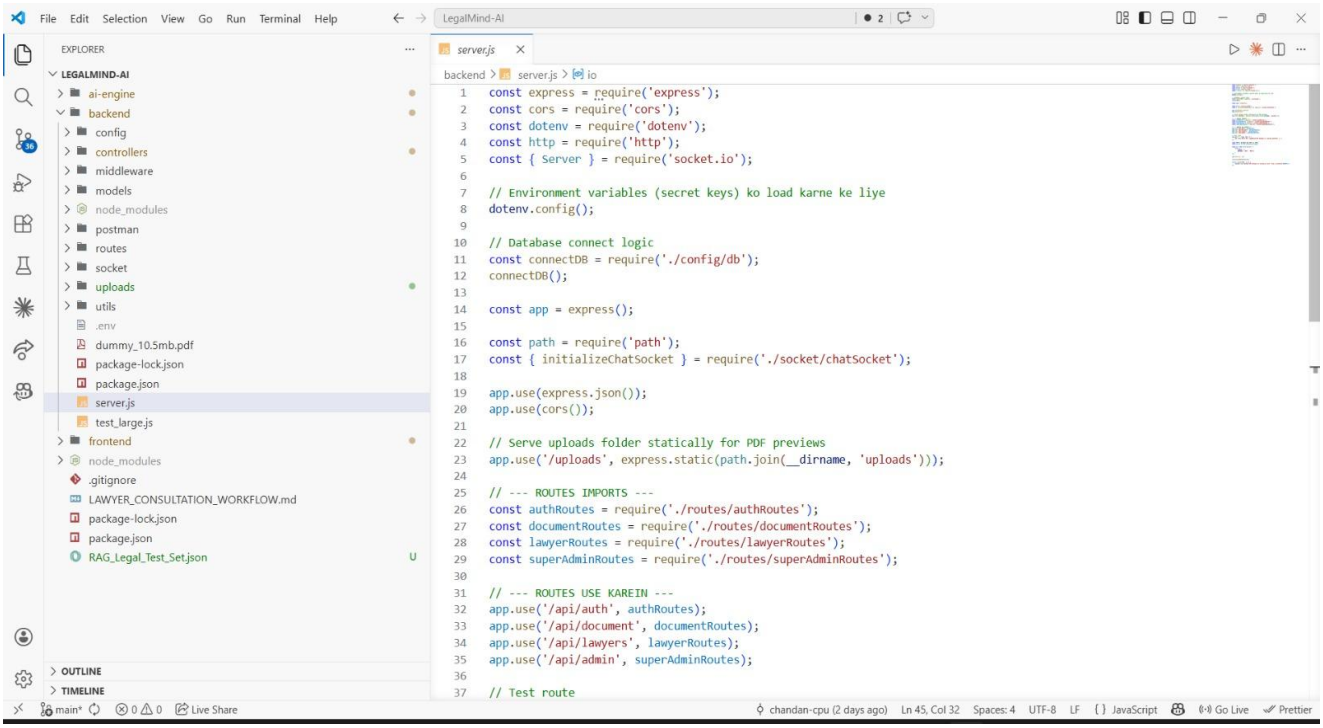
- Lawyer Registration & Verification
- View User Requests
- Accept/Reject Cases
- Provide Legal Feedback

Admin Module

- Approve Lawyers
- Manage Users
- Monitor System Activity

II . Code Snippet

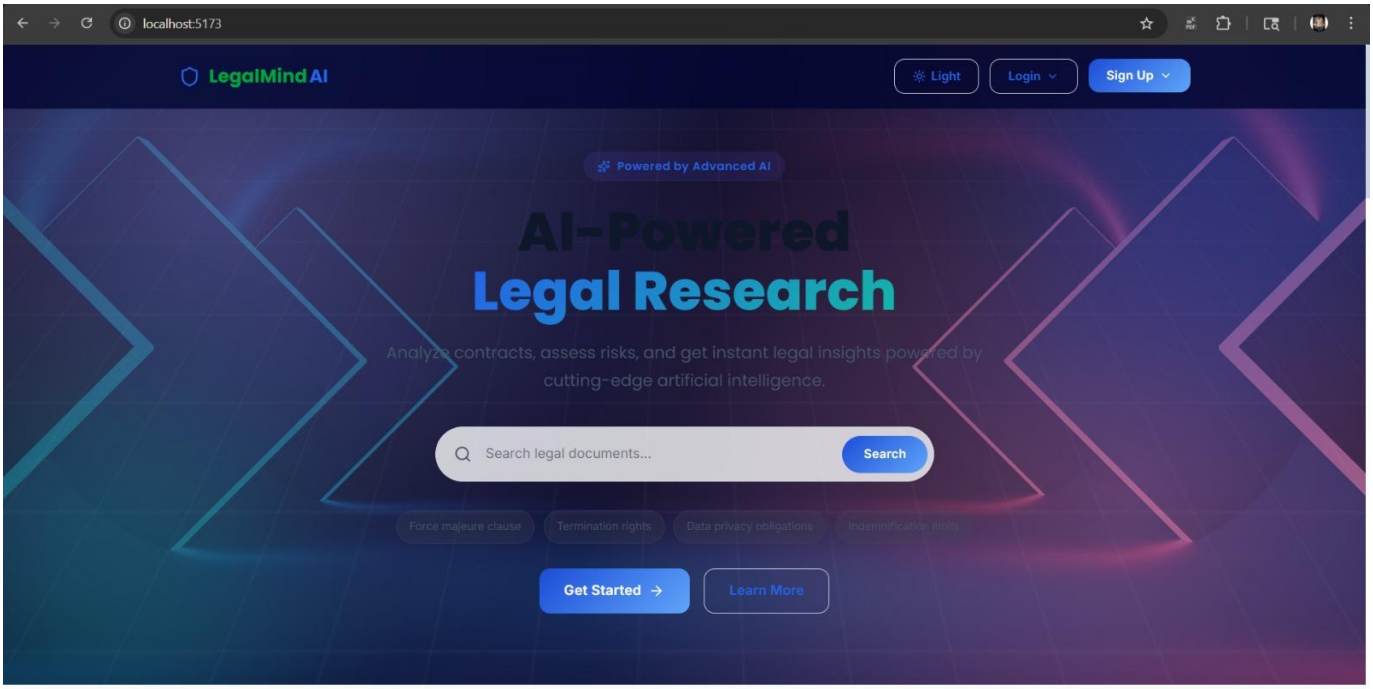


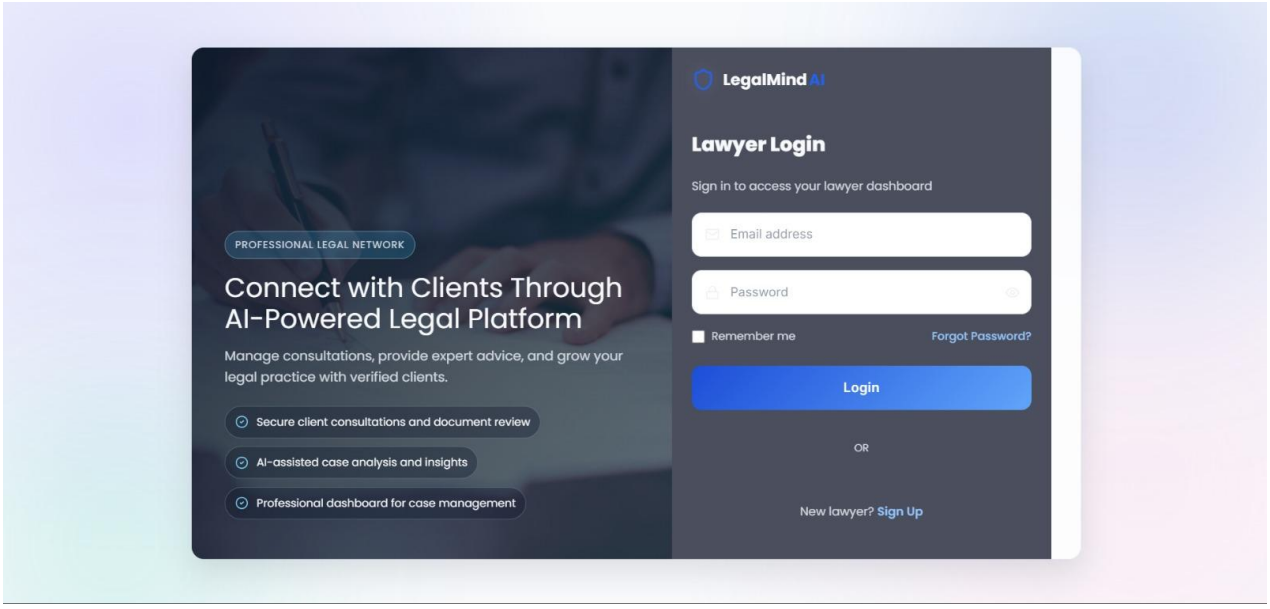


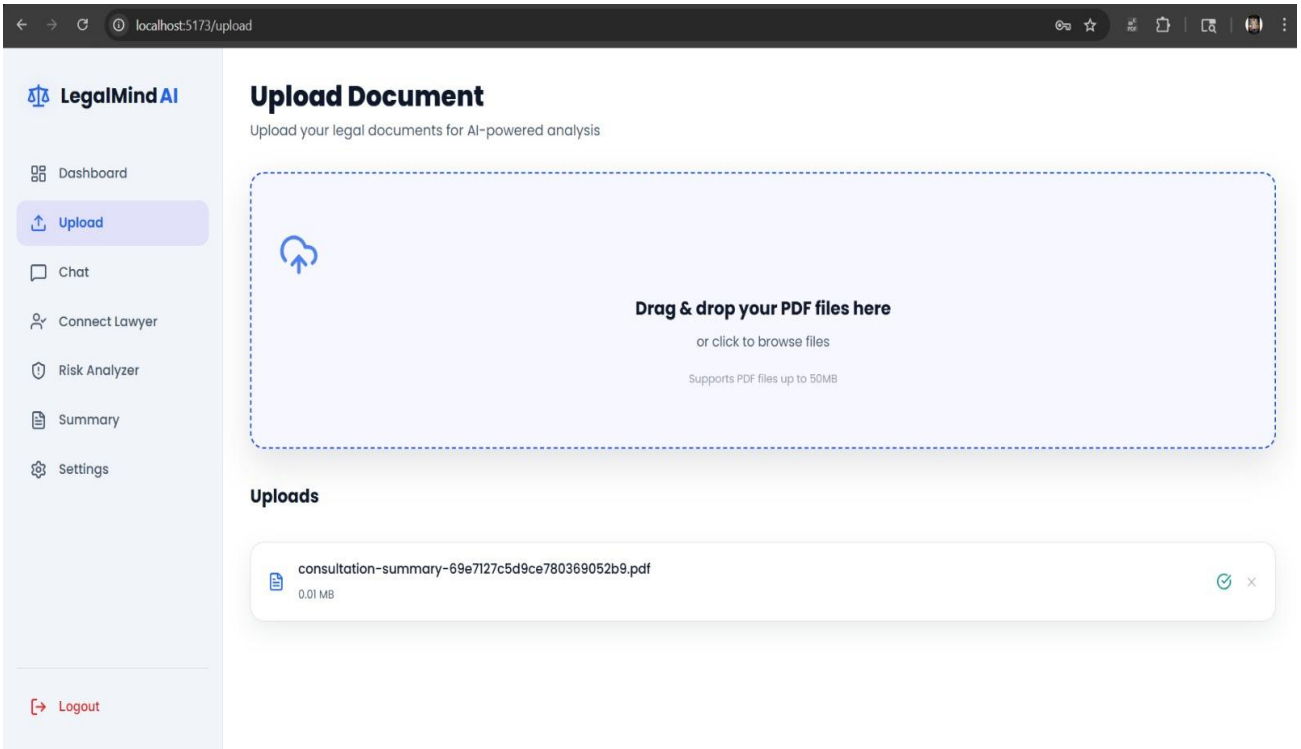
```python
# .env file se keys read karo
load_dotenv()

# Setup
embedder = SentenceTransformer('all-MiniLM-L6-v2')
pc = Pinecone(api_key=os.getenv("PINECONE_API_KEY"))
pinecone_index = pc.Index("legalmind-index")

def process_and_ingest_pdf_bytes(file_bytes: bytes, document_id: str):
    print(f"--> Processing doc: {document_id}")

    doc = fitz.open(stream=file_bytes, filetype="pdf")
    raw_text = ""
    for page in doc:
        raw_text += page.get_text()

    if not raw_text.strip():
        raise ValueError("No readable text found in PDF")

    # LangChain Text Splitter ka use karke smart chunking karo
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)

    # create_documents ki jagah split_text use karenge
    chunks_list = text_splitter.split_text(raw_text)

    if not chunks_list:
        raise ValueError("No text chunks generated from PDF")

    pinecone_vectors = []
    for i, chunk_text in enumerate(chunks_list):
        # Ab chunk_text seedha ek string hai, usko encode karo
        vector = embedder.encode(chunk_text).tolist()

        unique_id = f"{document_id}_chunk_{i}"
        metadata = {"document_id": document_id, "text": chunk_text}

        pinecone_vectors.append((unique_id, vector, metadata))
```

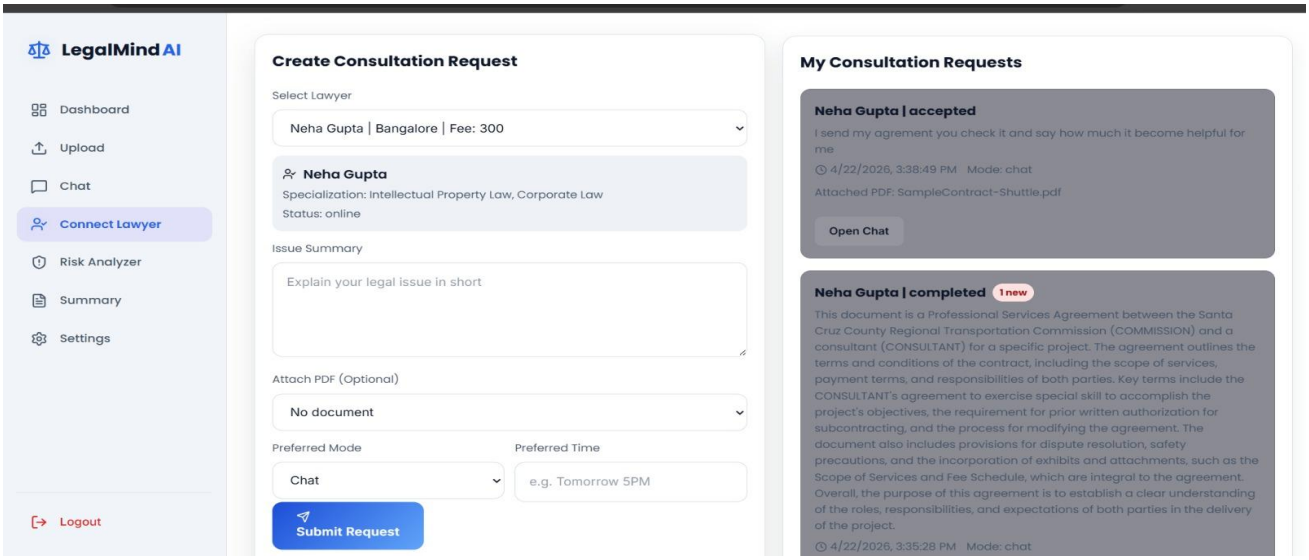
```json
[
    {
        "id": 1,
        "document": "Master Service Agreement",
        "question": "what is the specific definition of 'Confidential Information' according to the agreement?",
        "answer": "Confidential Information includes all information disclosed by one party to the other which i
        "section": "1.4"
    },
    {
        "id": 2,
        "document": "Master Service Agreement",
        "question": "What is the interest rate for late payments?",
        "answer": "Late payments accrue interest at a rate of 1.5% per month or the maximum rate permitted by la
        "section": "5.3"
    },
    {
        "id": 3,
        "document": "Master Service Agreement",
        "question": "What is the limitation of liability for claims arising out of this agreement?",
        "answer": "The total liability shall not exceed the amount paid by the User in the twelve (12) months pr
        "section": "9.2"
    },
    {
        "id": 4,
        "document": "Master Service Agreement",
        "question": "How many days notice are required to cure a material breach before termination?",
        "answer": "A party must fail to cure a breach within thirty (30) days of notice before the agreement car
        "section": "8.2"
    },
    {
        "id": 5,
        "document": "Executive Employment Agreement",
        "question": "How long is the probationary period for the employee?",
        "answer": "The first six (6) months of employment shall be a probationary period.",
        "section": "2.2"
    },
```

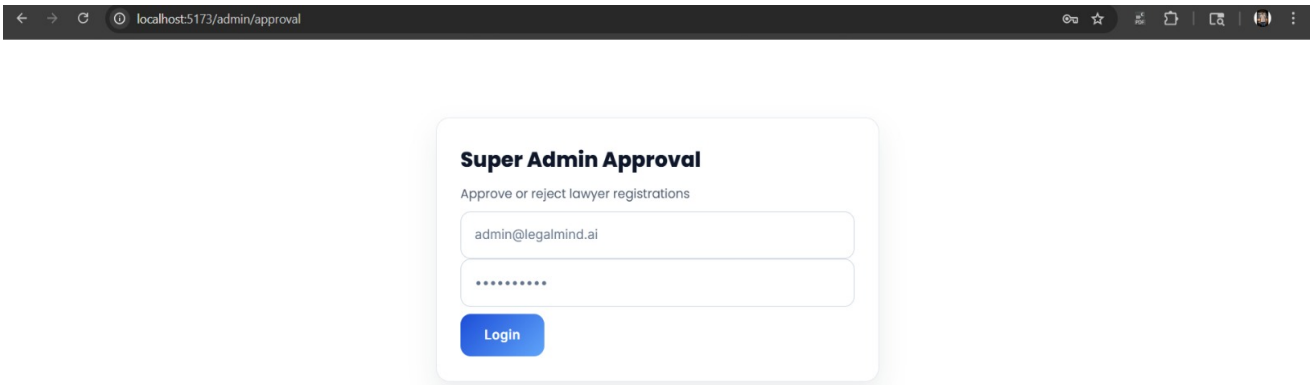
III . Output Screens

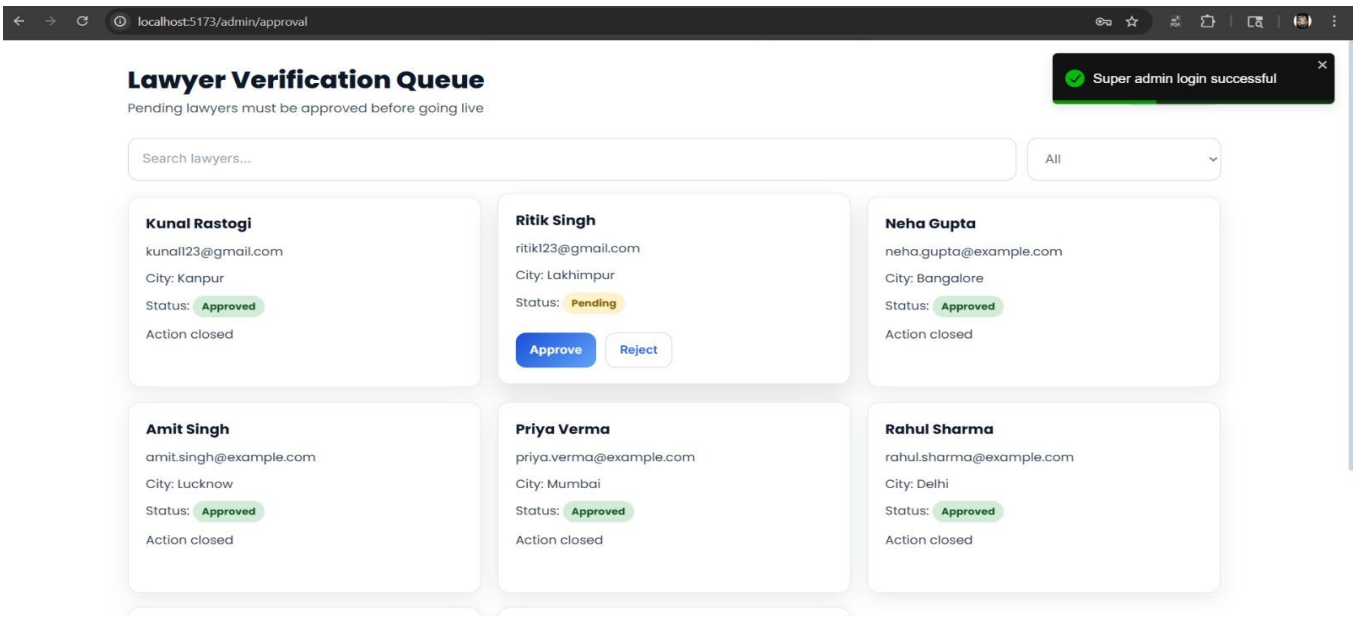


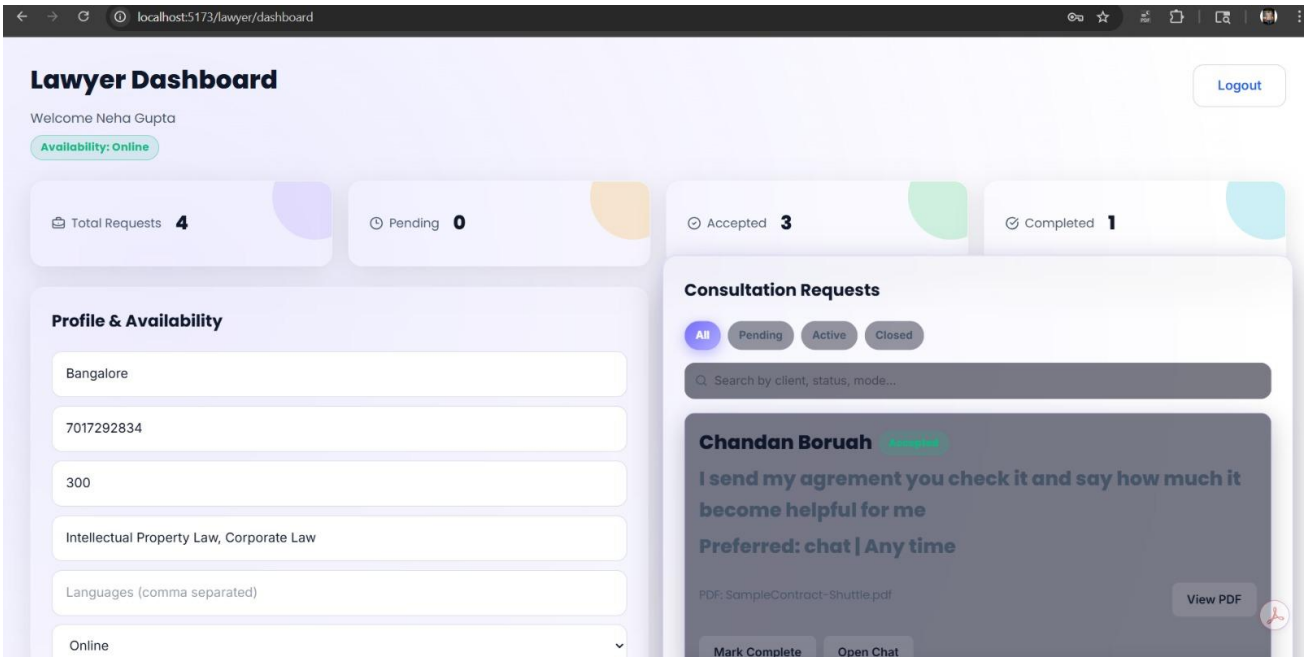



localhost:5173/upload
LegalMind AI
Dashboard
Upload
Chat
Connect Lawyer
Risk Analyzer
Summary
Settings
Logout
Upload Document
Upload your legal documents for AI-powered analysis
Drag & drop your PDF files here
or click to browse files
Supports PDF files up to 50MB
Uploads
consultation-summary-69e7127c5d9ce780369052b9.pdf
0.01 MB


LegalMind AI
Dashboard
Upload
Chat
Connect Lawyer
Risk Analyzer
Summary
Settings
Logout
Create Consultation Request
Select Lawyer
Neha Gupta | Bangalore | Fee: 300
Neha Gupta
Specialization: Intellectual Property Law, Corporate Law
Status: online
Issue Summary
Explain your legal issue in short
Attach PDF (Optional)
No document
Preferred Mode
Chat
Preferred Time
e.g. Tomorrow 5PM
Submit Request
My Consultation Requests
Neha Gupta | accepted
I send my agrement you check it and say how much it become helpful for me
4/22/2026, 3:38:49 PM Mode: chat
Attached PDF: SampleContract-Shuttle.pdf
Open Chat
Neha Gupta | completed
1 new
This document is a Professional Services Agreement between the Santa Cruz County Regional Transportation Commission (COMMISSION) and a consultant (CONSULTANT) for a specific project. The agreement outlines the terms and conditions of the contract, including the scope of services, payment terms, and responsibilities of both parties. Key terms include the CONSULTANT's agreement to exercise special skill to accomplish the project's objectives, the requirement for prior written authorization for subcontracting, and the process for modifying the agreement. The document also includes provisions for dispute resolution, safety precautions, and the incorporation of exhibits and attachments, such as the Scope of Services and Fee Schedule, which are integral to the agreement. Overall, the purpose of this agreement is to establish a clear understanding of the roles, responsibilities, and expectations of both parties in the delivery of the project.
4/22/2026, 3:35:28 PM Mode: chat


localhost:5173/admin/approval
Super Admin Approval
Approve or reject lawyer registrations
admin@legalmind.ai
Login


localhost:5173/admin/approval
Lawyer Verification Queue
Pending lawyers must be approved before going live
Super admin login successful
Search lawyers...
All
Kunal Rastogi
kunal123@gmail.com
City: Kanpur
Status: Approved
Action closed
Ritik Singh
ritik123@gmail.com
City: Lakhimpur
Status: Pending
Approve
Reject
Neha Gupta
neha.gupta@example.com
City: Bangalore
Status: Approved
Action closed
Amit Singh
amit.singh@example.com
City: Lucknow
Status: Approved
Action closed
Priya Verma
priya.verma@example.com
City: Mumbai
Status: Approved
Action closed
Rahul Sharma
rahul.sharma@example.com
City: Delhi
Status: Approved
Action closed


localhost:5173/lawyer/dashboard
Lawyer Dashboard
Logout
Welcome Neha Gupta
Availability: Online
Total Requests 4
Pending 0
Accepted 3
Completed 1
Profile & Availability
Bangalore
7017292834
300
Intellectual Property Law, Corporate Law
Languages (comma separated)
Online
Consultation Requests
All
Pending
Active
Closed
Search by client, status, mode...
Chandan Boruah
I send my agrement you check it and say how much it become helpful for me
Preferred: chat | Any time
PDF: SampleContract-Shuttle.pdf
View PDF
Mark Complete
Open Chat